

## **Laboratorio #5: Modelos de lenguaje**

### **Reporte escrito**

#### **1. Preparación del corpus para modelado de secuencias**

El corpus de Don Quijote quedó con 9,638 oraciones después de separar el texto y agregar los tokens de inicio y fin. Después se dividió de forma aleatoria en 7,710 oraciones para entrenamiento, 964 para validación y 964 para prueba, manteniendo la proporción de 80%, 10% y 10% que se pedía en el laboratorio.

El vocabulario del conjunto de entrenamiento tuvo 20,730 palabras diferentes. En prueba hubo 43,298 palabras en total y 1,371 no habían aparecido antes en entrenamiento, lo que representa un 3.17% de palabras OOV. Aunque el porcentaje no es demasiado alto, sí muestra el problema de data sparsity. Por más grande que sea el texto de entrenamiento, siempre pueden aparecer palabras o combinaciones nuevas.

Esto se vuelve todavía más importante cuando se pasa de palabras individuales a bigramas y trigramas, porque la cantidad de combinaciones posibles aumenta bastante y muchas aparecen pocas veces o nunca.

#### **2. Construcción de modelos n grama**

A partir del conjunto de entrenamiento se construyeron los tres modelos usando conteos. Se obtuvieron 20,731 unigramas diferentes, 124,671 bigramas y 248,328 trigramas. Solo con estos números ya se puede ver cómo aumenta la cantidad de combinaciones cuando se utiliza más contexto.

Para la oración de validación "no, por cierto respondió el labrador.", el unigrama obtuvo una probabilidad aproximada de  $7.39e-21$ , el bigrama de  $8.62e-13$  y el trigrama de  $9.12e-10$ . En este ejemplo el trigrama dio la probabilidad más alta porque utiliza las dos palabras anteriores y tiene más información del contexto local.

La regla de la cadena permitiría calcular la probabilidad de una oración usando todo lo que apareció antes de cada palabra. El problema es que en una oración real esto genera demasiadas combinaciones y muchas nunca se observan en entrenamiento. El supuesto de Markov hace el cálculo más manejable porque limita el contexto. El bigrama usa una palabra anterior y el trigrama usa dos. Se pierde parte del contexto completo, pero a cambio sí es posible estimar las probabilidades con los datos disponibles.

### 3. Suavizado

La oración "¡bendito sea dios!" tuvo probabilidad 0 con el modelo de bigramas sin suavizado. Esto pasó porque al menos uno de sus bigramas no apareció en entrenamiento. Como las probabilidades de la oración se multiplican, con que una sola sea cero toda la oración termina con probabilidad cero.

Después de aplicar suavizado, la probabilidad dejó de ser cero. Con Laplace usando  $k$  igual a 1 fue aproximadamente  $1.31e-17$ . Con  $k$  de 0.1 fue  $1.21e-14$  y con  $k$  de 0.01 fue  $9.57e-13$ . Lo importante no es que estas probabilidades sean grandes, sino que el modelo ya no considera imposible una combinación solamente porque nunca la vio.

El suavizado le quita una pequeña parte de probabilidad a las combinaciones observadas para poder darle algo a las no vistas. Esto ayuda a que el modelo no tenga demasiada confianza en que el conjunto de entrenamiento contiene todas las posibilidades.

### 4. Evaluación con perplejidad

Los resultados de validación fueron bastante claros. El unigrama obtuvo una perplejidad de 621.97, el bigrama de 427.04 y el trigramas de 2,326.32. Como una perplejidad menor significa que el modelo está prediciendo mejor el texto, el bigrama fue el que mejor funcionó.

Al principio se podría pensar que el trigramas debería ganar porque utiliza más contexto, pero aquí pasó lo contrario. El unigrama tiene muy poco contexto, mientras que el trigramas tiene demasiadas combinaciones específicas y muchas aparecen pocas veces. El bigrama quedó en un punto medio y logró un mejor equilibrio. Por esa razón fue elegido como modelo final. En el conjunto de prueba obtuvo una perplejidad de 465.33, bastante cercana a la de validación, por lo que su comportamiento se mantuvo parecido con datos nuevos.

También se vio bastante el efecto del valor de  $k$ . Con  $k$  igual a 0.01 la perplejidad del bigrama fue 427.04, con 0.1 subió a 688.37 y con 1 llegó a 1,799.43. En este caso aumentar demasiado el suavizado terminó empeorando el modelo porque se estaba repartiendo demasiada probabilidad hacia combinaciones poco frecuentes o no vistas. Por eso el valor de 0.01 dio el mejor resultado.

## 5. Aplicación práctica: autocompletado

El autocompletado se hizo con el bigrama porque fue el modelo con menor perplejidad. Los resultados fueron mixtos, pero sí se ve que aprendió varias relaciones útiles entre palabras. En "en el espacio", la palabra real era "que" y apareció como segunda sugerencia. En "viéndole, pues, caído", la palabra real era "en" y el modelo la colocó en primer lugar. En "cardenio estaba en", la palabra correcta era "el" y apareció en segundo lugar, apenas detrás de "la".

En los otros dos ejemplos el modelo no acertó la palabra real dentro de las primeras cinco opciones. Después de "pero, en fin, tanto" la palabra real era "hicieron", y después de "yo así lo" era "creo". Esto tiene sentido porque el bigrama solamente mira la última palabra y no entiende toda la idea de la oración como lo haría una persona. También apareció la palabra "mesmo", que refleja el vocabulario de Don Quijote. Como el modelo aprendió únicamente de esta novela, es normal que sus sugerencias tengan un estilo diferente al español actual.

## 6. Análisis

**¿Por qué la regla de la cadena de probabilidad, aunque es exacta, no se puede utilizar directamente para estimar la probabilidad de una oración con datos reales?**

El problema es que cada nueva palabra tendría que condicionarse a todo lo que apareció antes. Mientras más larga sea la oración, más específico se vuelve ese historial y menos probable es que exactamente esa combinación haya aparecido en entrenamiento. Esto se conecta directamente con el 3.17% de OOV encontrado en prueba.

Si incluso con palabras individuales aparecen términos nunca vistos, al considerar secuencias completas el problema se hace mucho mayor. Por eso los modelos n grama simplifican el historial y permiten trabajar con cantidades de datos más realistas.

**¿Los resultados confirman el trade off entre contexto y dispersión de datos?**

Sí, en este laboratorio se ve bastante claro. El unigrama es fácil de estimar, pero no usa contexto y terminó con una perplejidad de 621.97. El bigrama agregó algo de contexto y mejoró a 427.04. El trigramma usa todavía más contexto, pero su perplejidad subió hasta 2,326.32 porque las combinaciones de tres palabras son mucho más dispersas. En generación pasa algo parecido.

El unigrama tiende a proponer palabras muy frecuentes sin importar mucho el fragmento, mientras que un trigramma puede dar sugerencias más específicas cuando conoce bien ese contexto, pero falla más cuando la combinación es rara. En este corpus el bigrama terminó dando el balance más estable entre las dos cosas.

### **¿Qué efecto tuvo el suavizado sobre la perplejidad y sobre la calidad de las predicciones?**

El suavizado fue necesario para evitar que una combinación no vista hiciera que toda la oración tuviera probabilidad cero. Eso permitió calcular la perplejidad sobre validación y prueba sin tratar como imposibles todas las secuencias nuevas. Sin embargo, también se vio que más suavizado no significa automáticamente un mejor modelo. Cuando  $k$  aumentó de 0.01 a 1, la perplejidad del bigrama pasó de 427.04 a 1,799.43.

Esto muestra que el suavizado ayuda cuando se usa con cuidado, pero si se exagera se le quita demasiado peso a los patrones que sí aprendió el modelo. Es necesario incluso con un corpus grande porque siempre van a existir palabras o combinaciones que no aparecieron en entrenamiento.

### **Si tuviera que elegir un valor de $n$ para un sistema de autocompletado en producción, ¿cuál elegiría?**

Para este caso elegiría un bigrama. Fue el modelo con menor perplejidad y además necesita guardar menos combinaciones que el trigramma, 124,671 frente a 248,328. Eso significa menos memoria y cálculos más simples, sin perder toda la información del contexto como pasa con el unigrama.

Un modelo más grande podría llegar a producir sugerencias más específicas en algunos casos, pero con los datos de este laboratorio el trigramma no compensó ese costo. Por velocidad, memoria y calidad, el bigrama sería la opción más razonable.

Link para el github: <https://github.com/luispegr25/NLP>